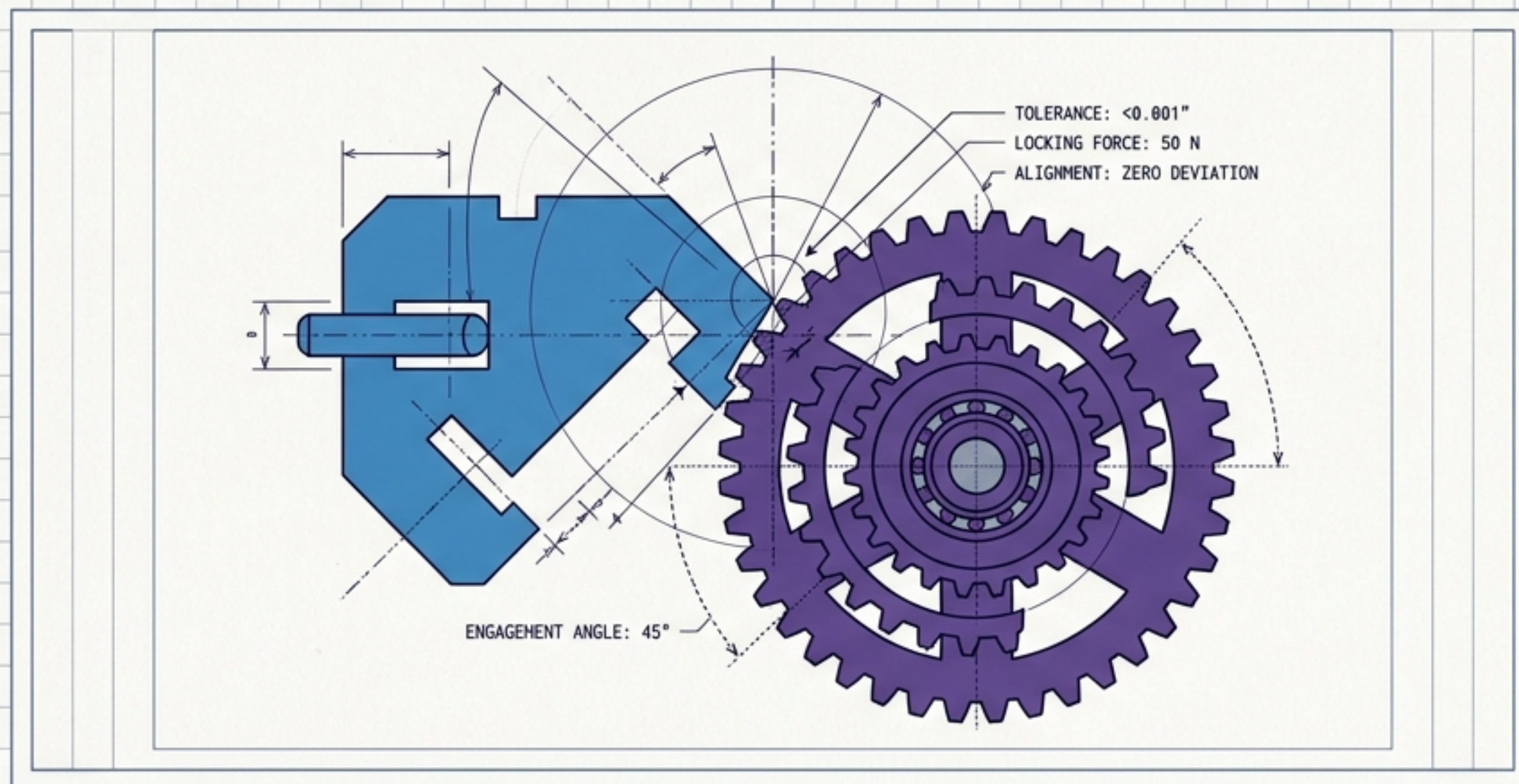


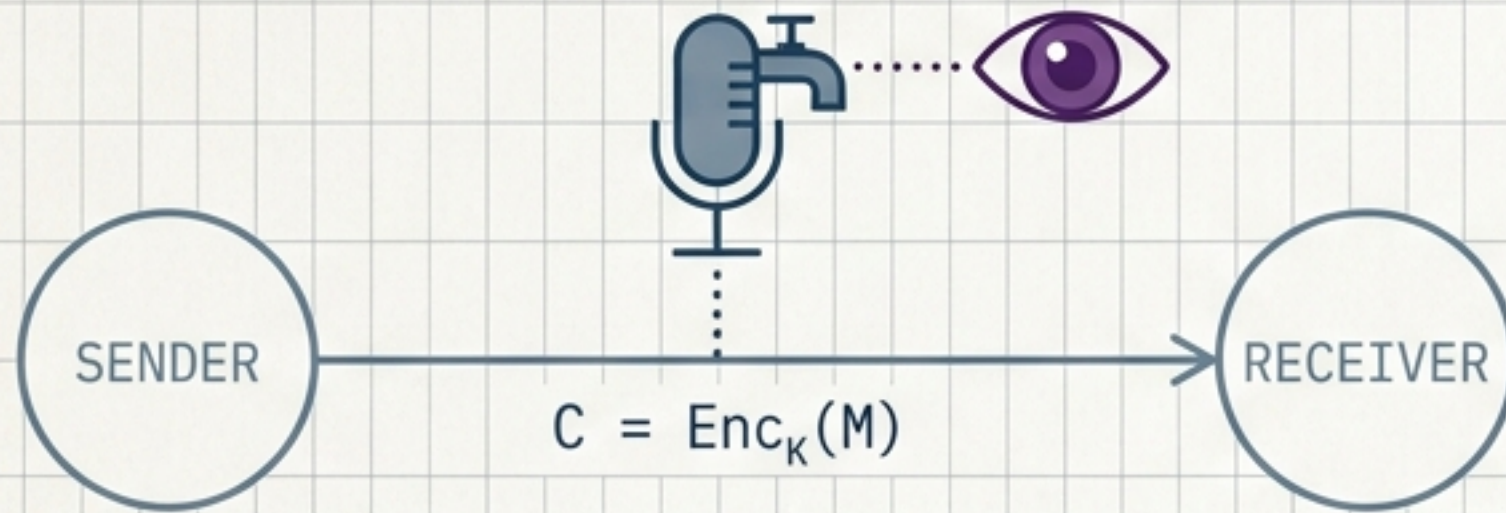
The Architecture of Integrity

Constructing Message Authentication Codes and Hash Functions



SPECIFICATION_REF: CH_03 // PRIMITIVES & COMPOSITION

The Passive Adversary



Encryption hides data. It succeeds against passive observation.

The Active Adversary



The attacker does not need the key K to manipulate the ciphertext and produce meaningful, dangerous effects after decryption.

Property	Mechanism	Core Goal	Failure State
Confidentiality	$C = Enc_K(M)$	Prevent unauthorized disclosure (Hiding)	An attacker reads the message.
Integrity	$T = MAC_K(M)$	Prevent undetected modification (Authenticating)	An attacker forges a message.

The Bank Example



Encryption hides "Transfer \$100". Without integrity, an attacker blindly flips bits to create "Transfer \$10,000". If accepted, confidentiality succeeded, but integrity failed.

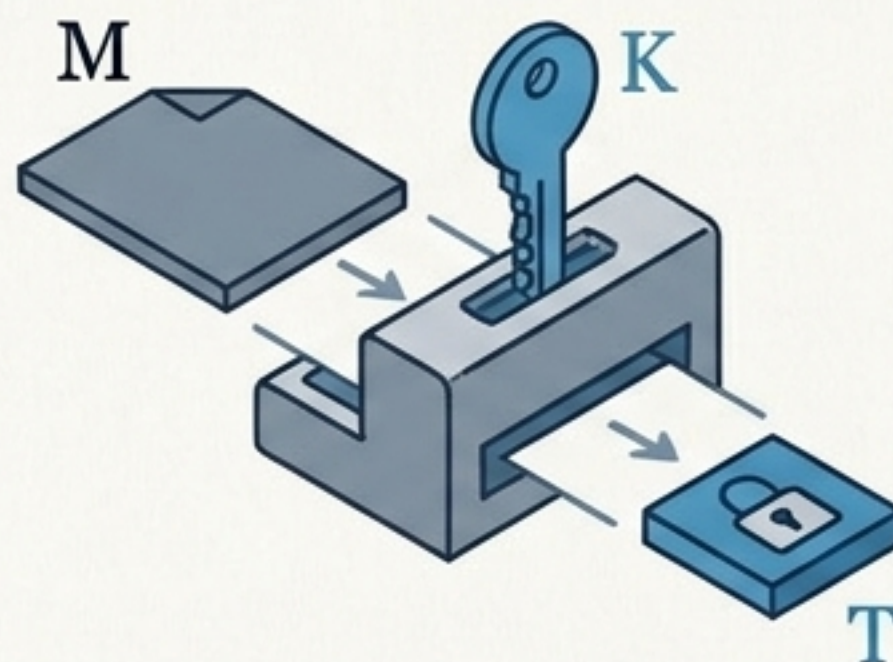
Step 1: Generate

$$K \leftarrow \text{Gen}(1^\lambda)$$



Step 2: Tag

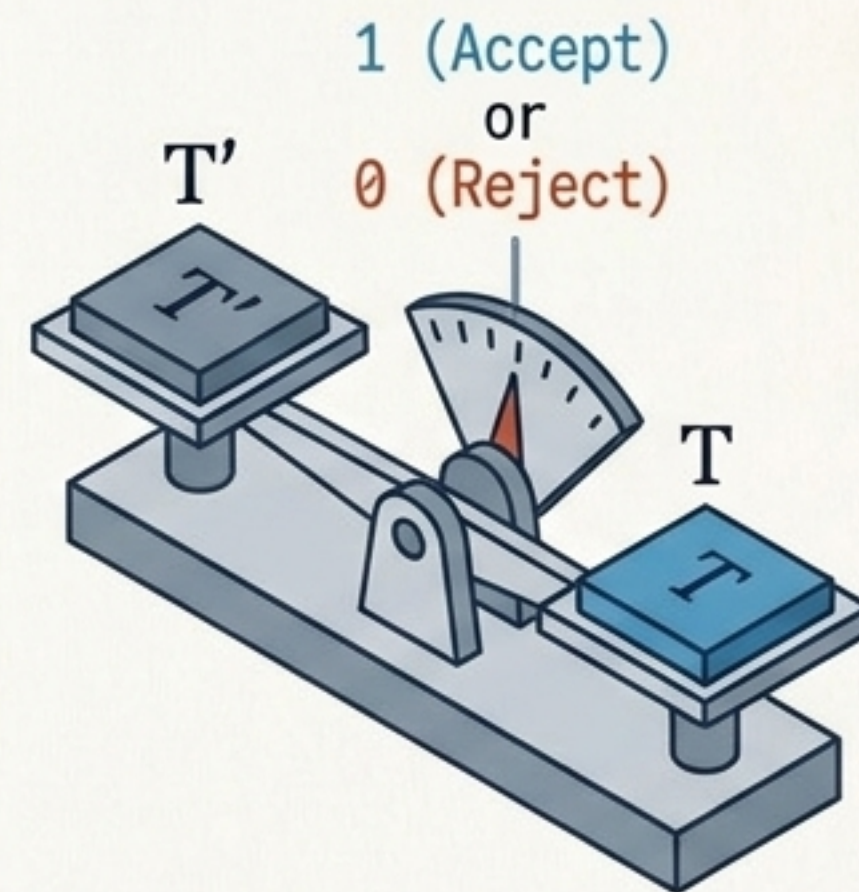
$$T \leftarrow \text{Tag}_K(M)$$



Produces a compact cryptographic authenticator.

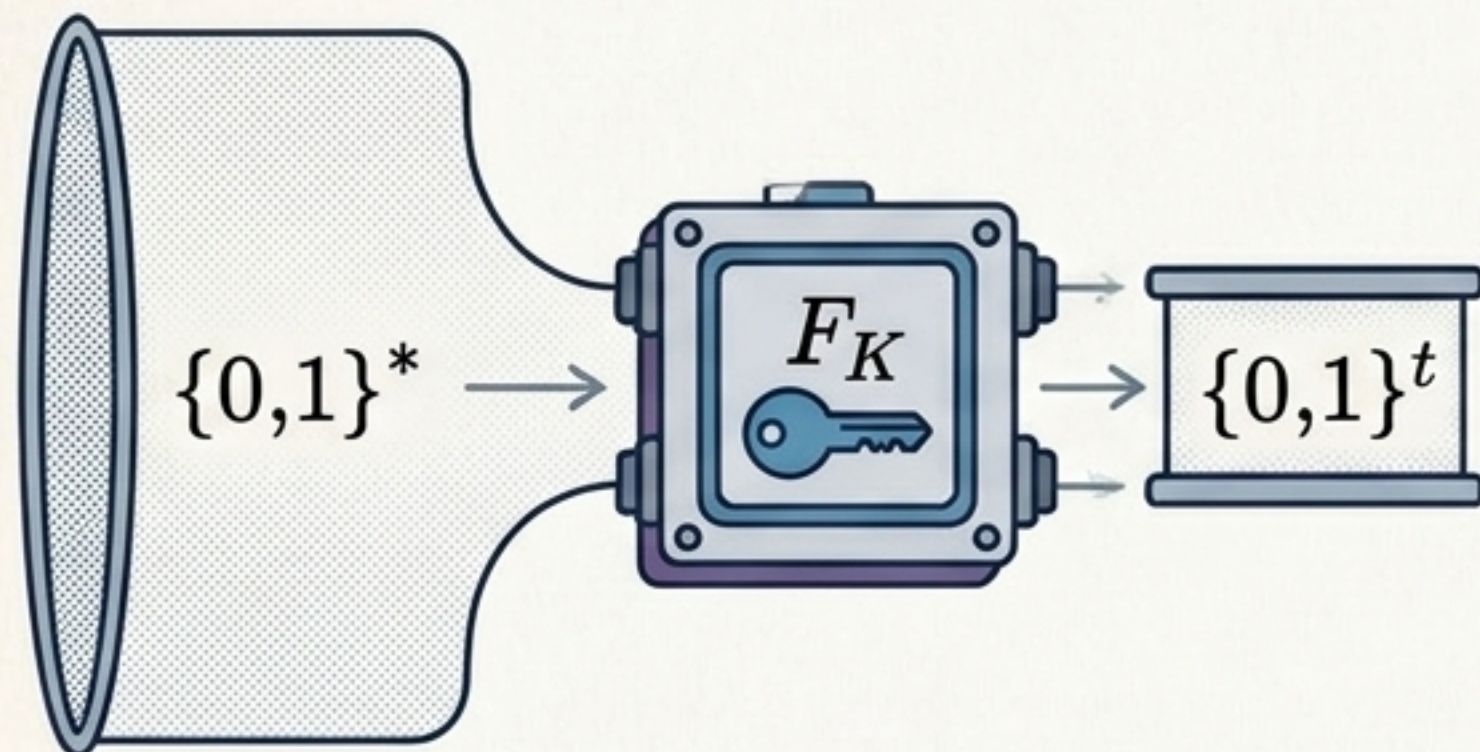
Step 3: Verify

$$b \leftarrow \text{Verify}_K(M, T)$$



Unlike a standard checksum designed for accidental errors, a MAC resists deliberate manipulation. It works solely because the attacker lacks the secret key K .

The PRF Engine

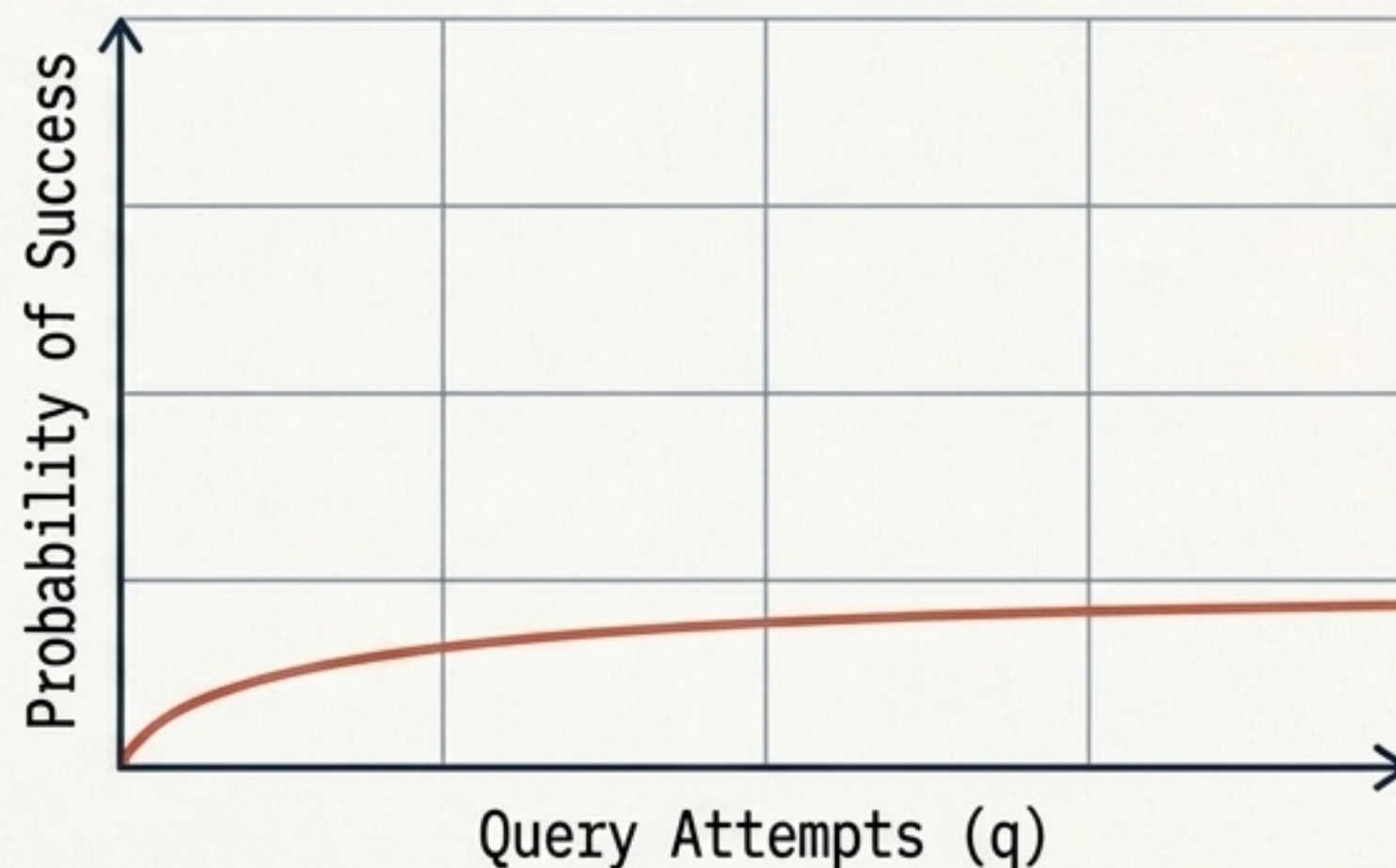


A secure PRF behaves like a random function that only the key holder can evaluate.

The Math of Forgery

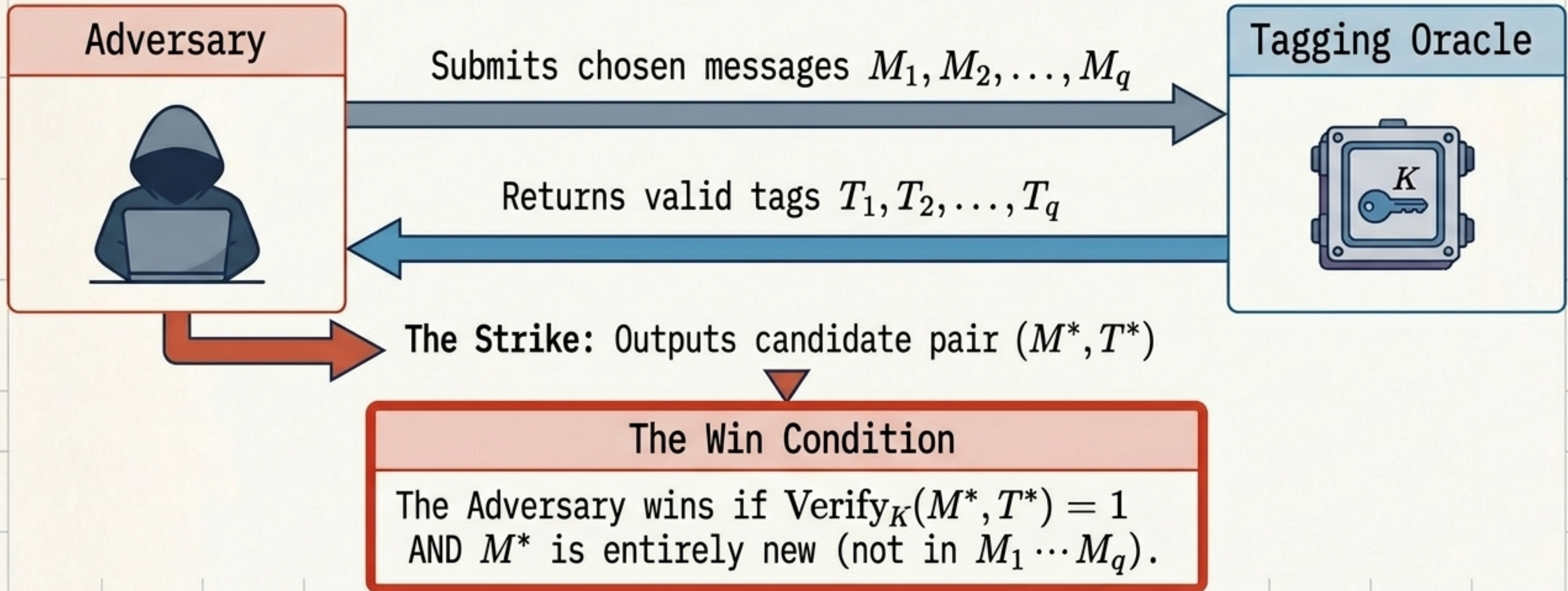
Success probability of a single guess: $\sim 2^{-t}$

Cumulative probability after q attempts: $\leq \frac{q}{2^t}$



Security relies on sufficient tag length (t) and rate-limiting query attempts (q). Short tags become dangerous under repeated probing.

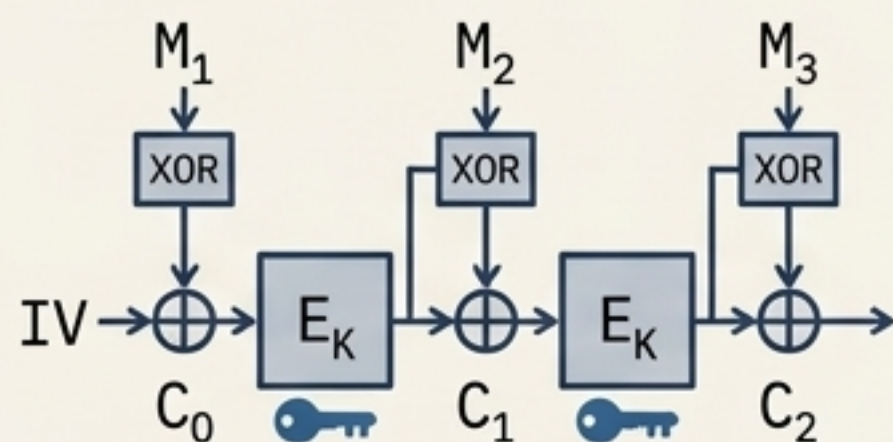
The Forgery Game: Unforgeability under Chosen-Message Attack



Existential forgery means ANY valid new message is a catastrophic break. The attacker does not need to forge a specific, meaningful message to win.

Evolution of Arbitrary-Length Message Handling

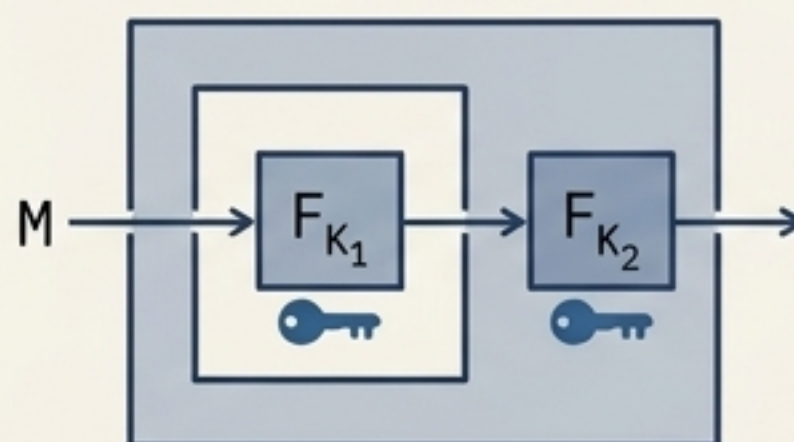
CBC-MAC (Sequential)



$$C_i = E_K(M_i \text{ XOR } C_{i-1})$$

Note:
Elegant, but vulnerable to extension attacks if message length varies. Secure only for fixed-length inputs.

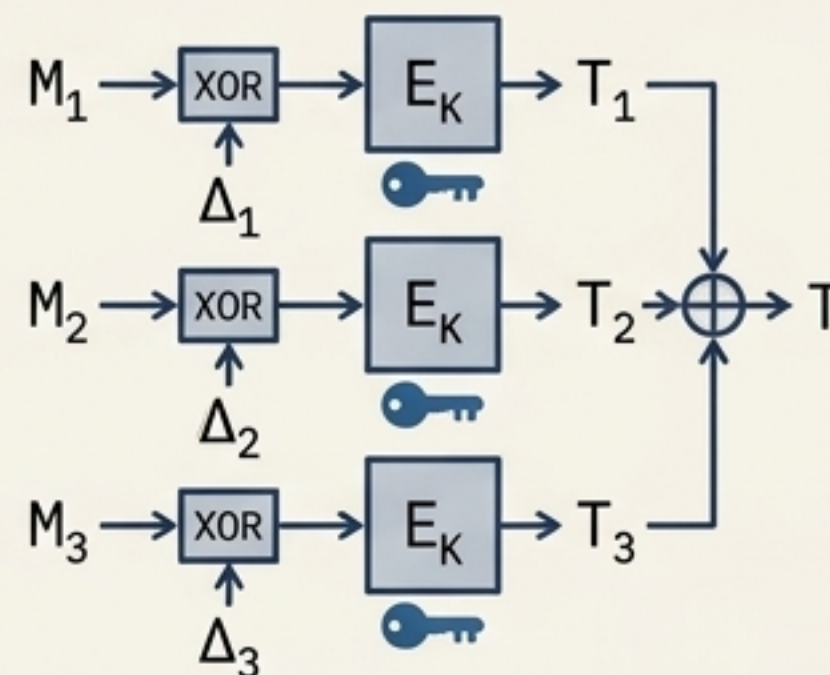
NMAC (Nested)



$$F_{K_2}(F_{K_1}(M))$$

Note:
The theoretical bridge. Proves that nesting prevents extension vulnerabilities.

PMAC (Parallel)



$$E_K(M_i \text{ XOR } \Delta_i)$$

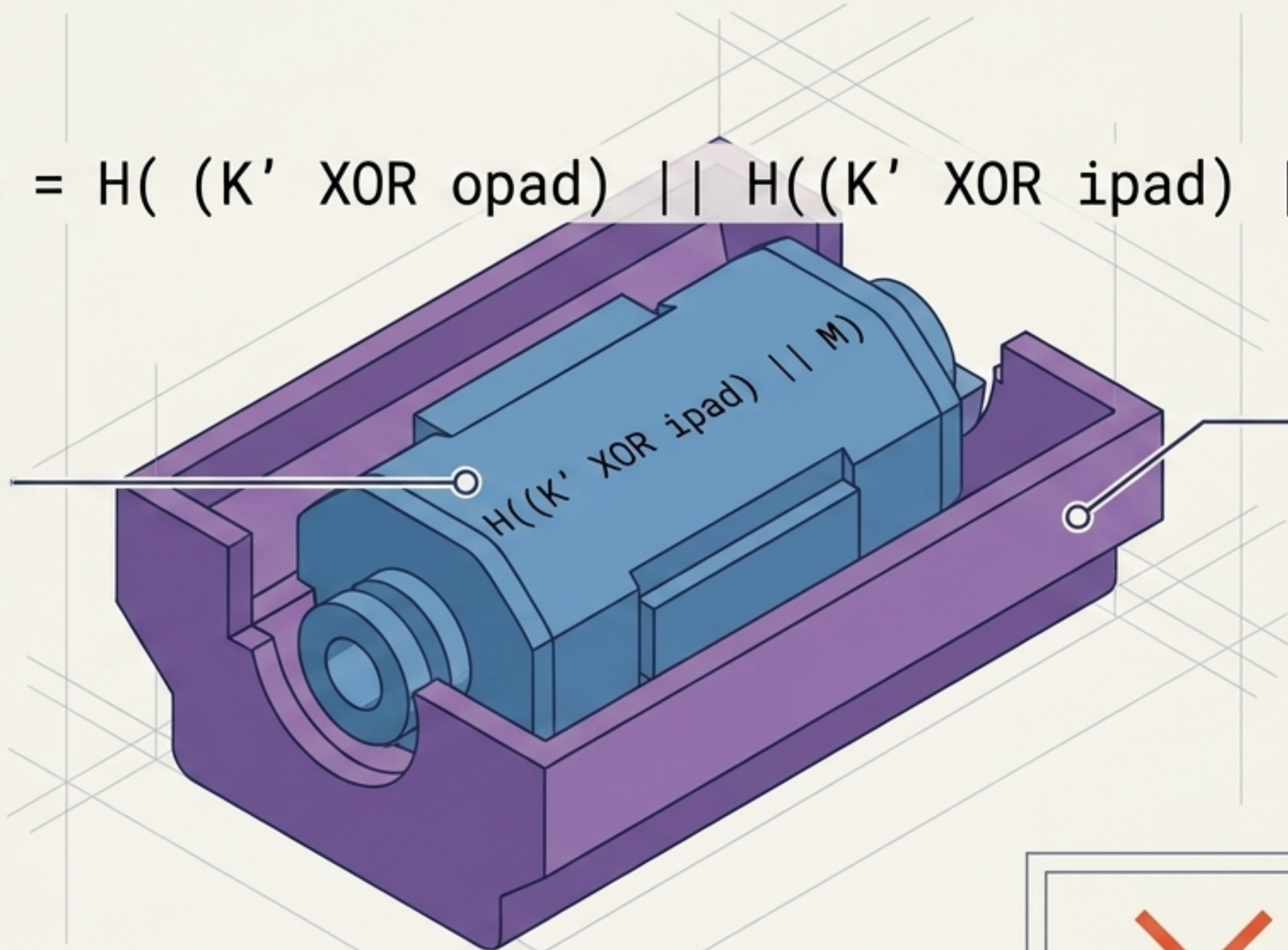
Note:
Overcomes sequential bottlenecks, optimized for high-speed hardware processing.

A construction secure for fixed-length messages is not automatically secure for arbitrary-length messages.

The Industry Standard: HMAC Exploded View

$$\text{HMAC} = H((K' \text{ XOR opad}) \parallel H((K' \text{ XOR ipad}) \parallel M))$$

Inner Core:
 $H((K' \text{ XOR ipad}) \parallel M)$
Inner boundary: Hashes
the message with the
inner padded key.

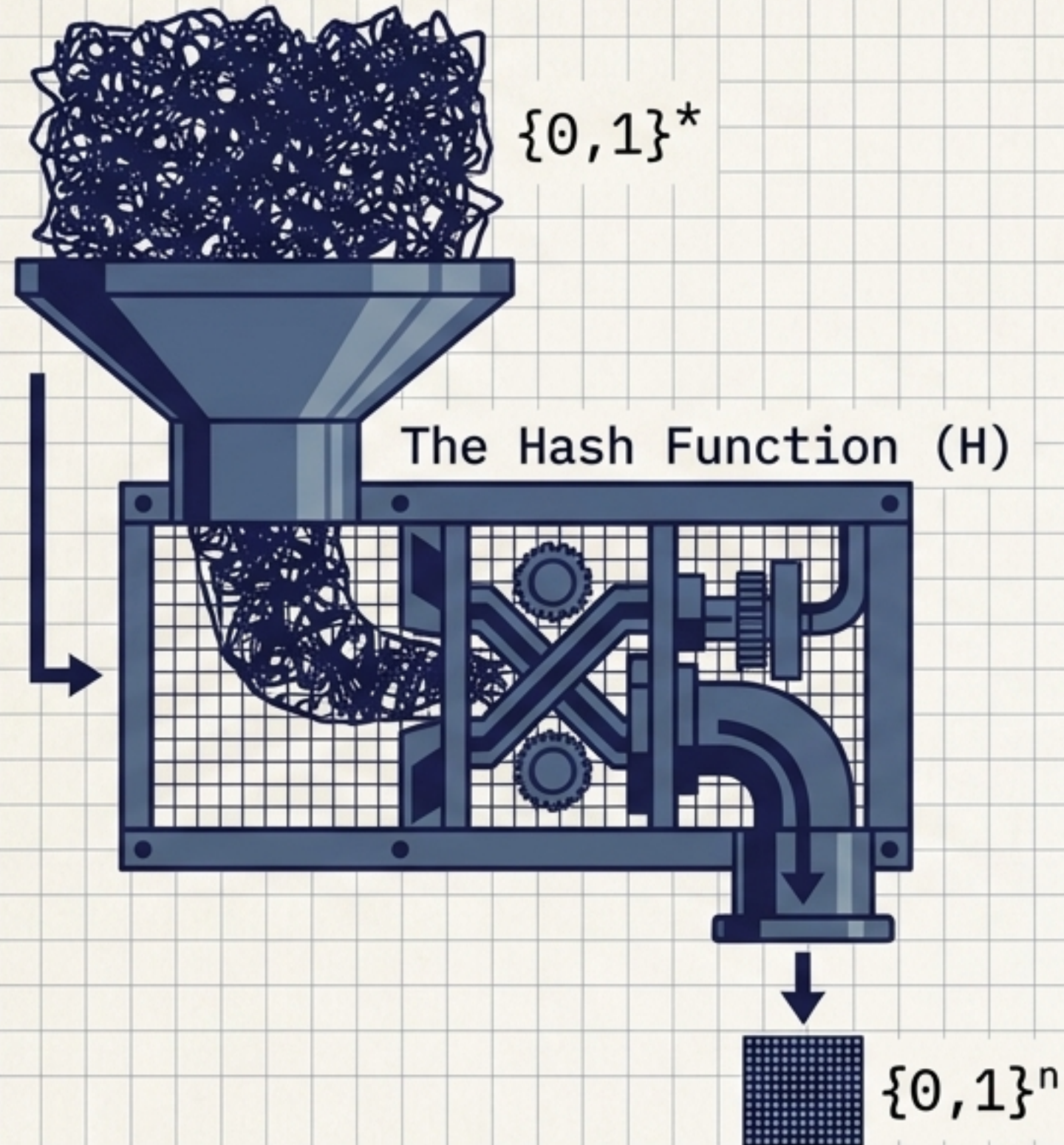


Outer Shell:
 $H((K' \text{ XOR opad}) \parallel$
inner result)
Outer boundary:
Hashes the result with
the outer padded key.

~~$H(K \parallel M)$~~

HMAC explicitly avoids
simple concatenation, which
fails catastrophically
against internal structural
weaknesses of hash functions.

The Hash Function (H): A One-Way Compressor



Deterministic: Same input always yields the exact same digest.

Unkeyed: No secret key. Anyone can compute $H(M)$.

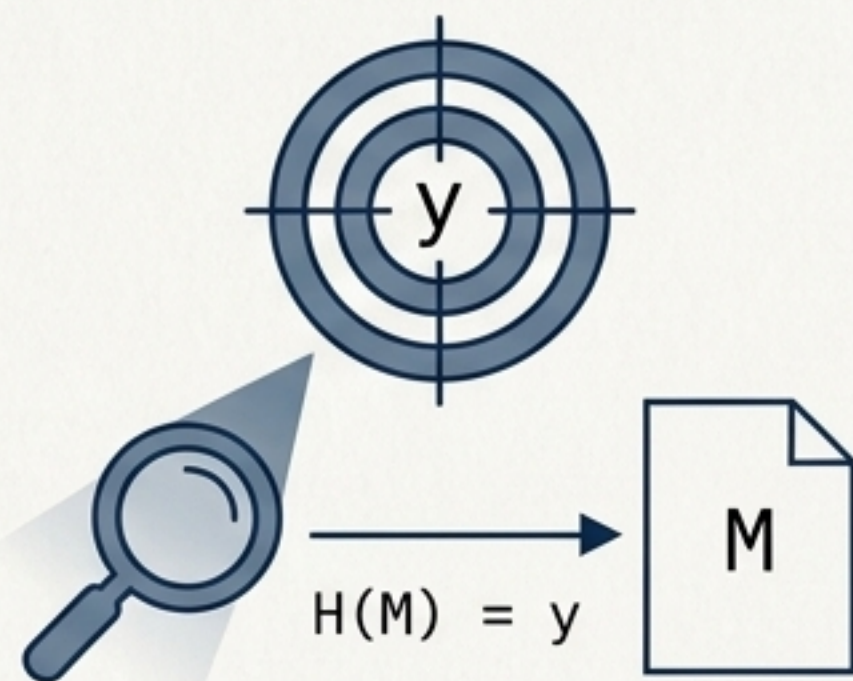
One-Way: No efficient decryption algorithm exists.



Hash $\neq$ MAC. A hash function by itself does NOT authenticate a message, because any attacker can compute a new hash for an altered message.

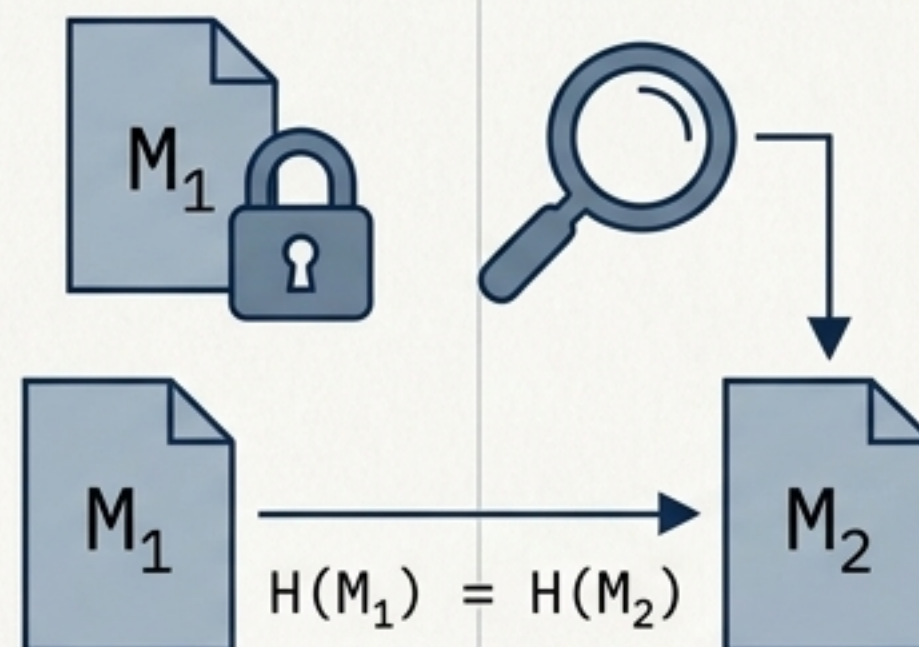
Hash Security: Preimage, Second-Preimage, and Collision Resistance

Preimage Resistance



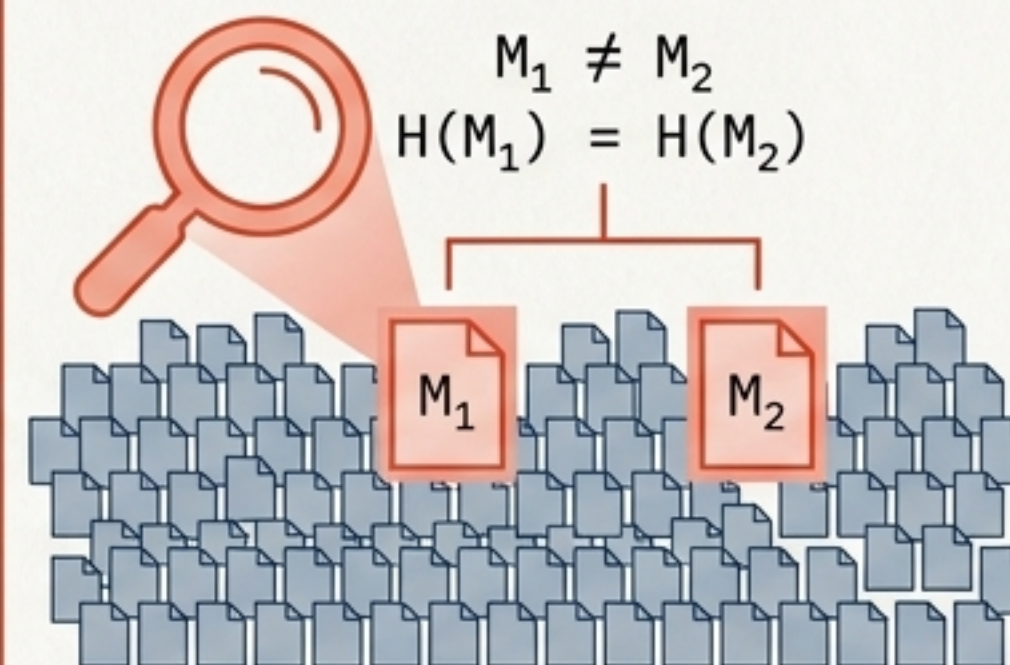
Complexity: 2^n
Full search space required.

Second-Preimage Resistance



Complexity: 2^n
Full search space required.

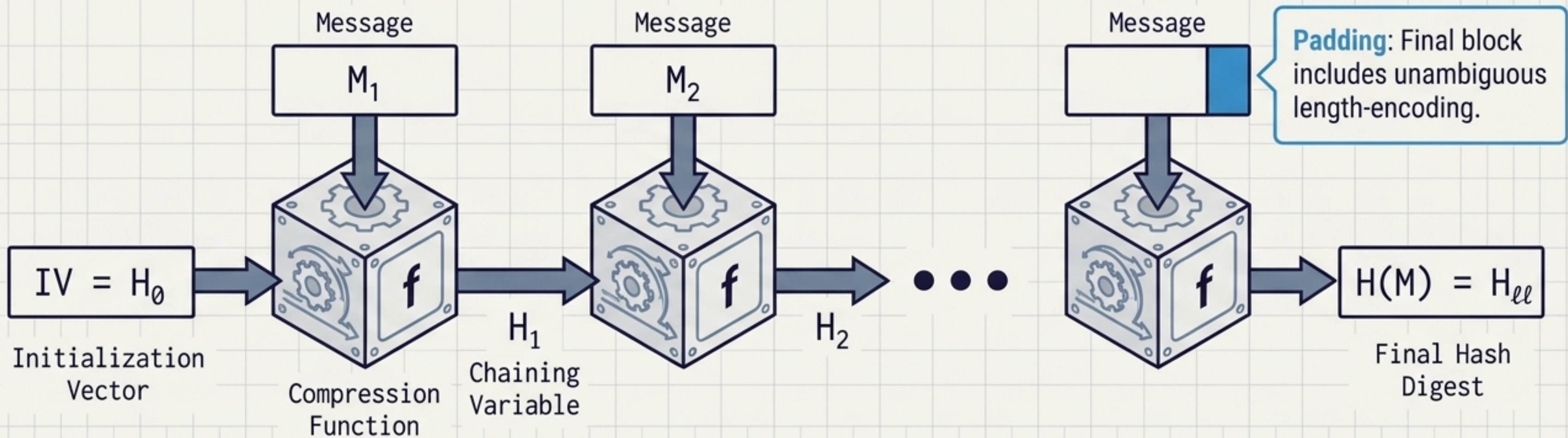
Collision Resistance (The Weakest Link)



Complexity: $2^{n/2}$
The Birthday Paradox.

Because of the Birthday Paradox, an ideal 256-bit hash offers 2^{256} preimage security, but only 2^{128} collision security. Collision resistance dictates the true security bound.

The Merkle-Damgård Construction: A Chained Iterative Architecture



Modern alternatives like SHA-3 use a fundamentally different 'Sponge' architecture (rate/capacity) to avoid the structural quirks inherent to this chaining paradigm.

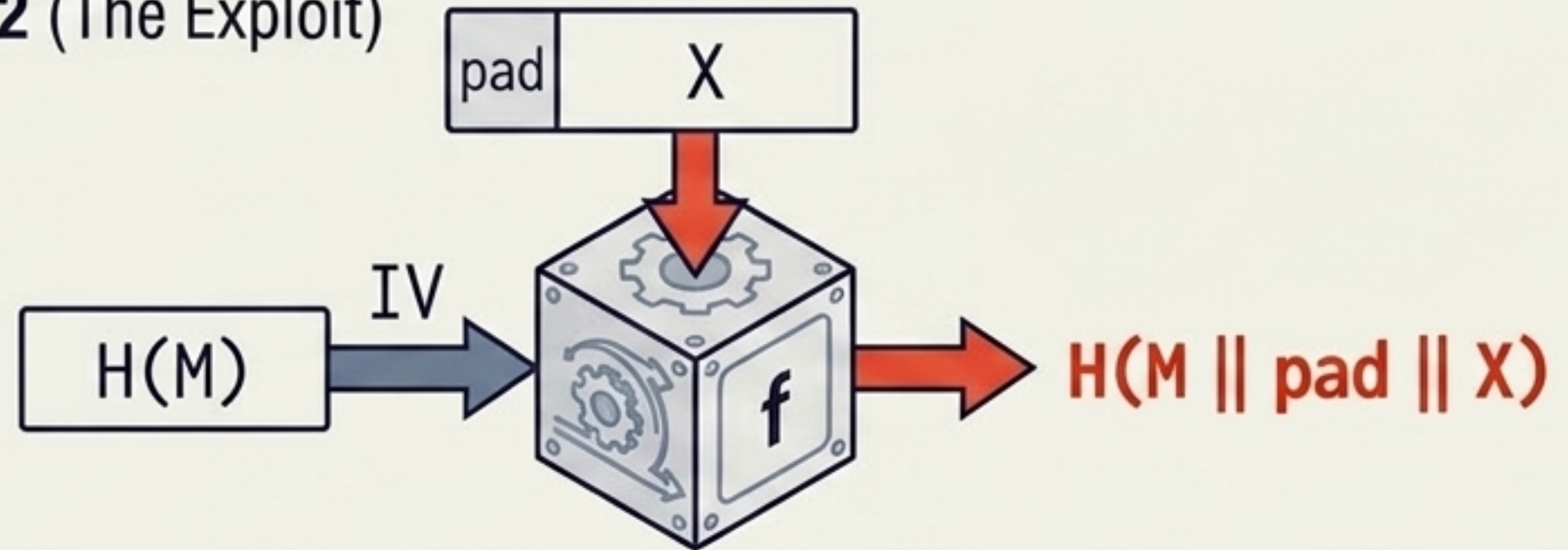
Structural Vulnerability: The Length-Extension Attack

Step 1 (The Setup)

$H(M)$

Attacker captures a valid hash. They do NOT know the message M , but they know its length.

Step 2 (The Exploit)



Because $H(M)$ is just the internal state (H_{ell}) of the chain, the attacker successfully extends the message and computes a valid new hash.

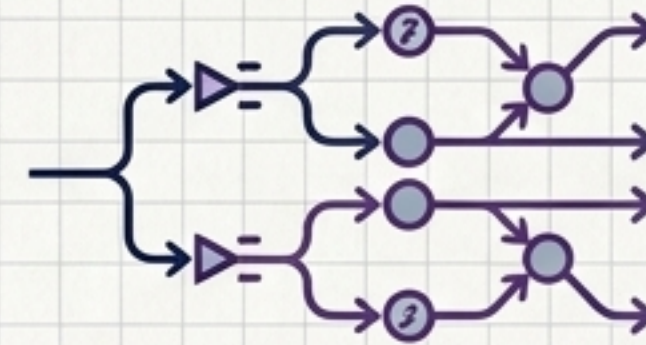
This is a structural property, not a broken hash. It is the exact reason naive MACs like $T = H(K \parallel M)$ fail instantly—the attacker simply extends the message and calculates a valid tag without needing the secret key.

Brute-Force & Truncation



Attackers guess tags with probability $\frac{q}{2^t}$. Severe risk if tags are truncated or failed attempts aren't rate-limited.

Differential Cryptanalysis



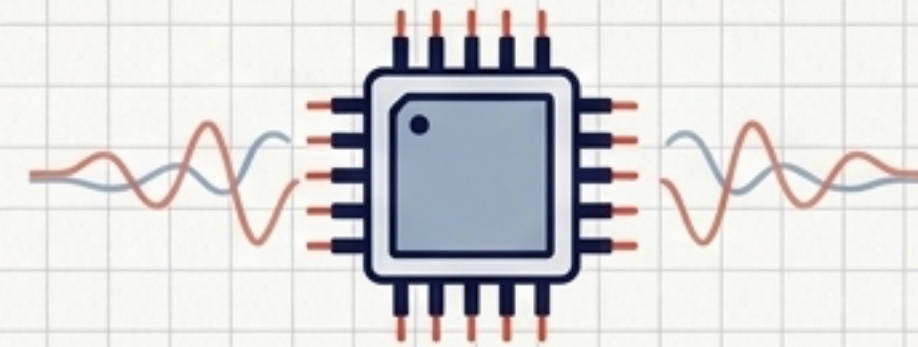
Exploiting how controlled differences in input propagate through internal nonlinear transformations to find favorable cancellation paths.

Structural Composition



Using CBC-MAC for variable-length data, or $H(K \parallel M)$. The underlying primitives are mathematically secure, but their domain assumptions are violated.

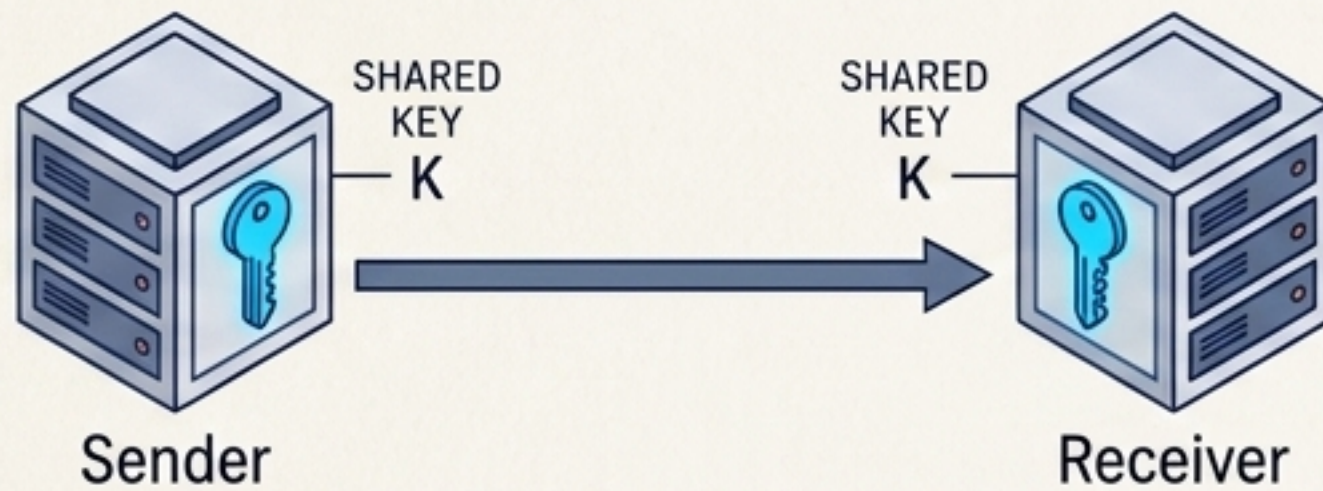
Side-Channel Leaks



The math is perfect, but the physical implementation leaks. Early-exit string comparisons leak timing data; power consumption reveals the secret key.

MAC Limits: Provenance vs. Contextual Integrity

What a MAC Proves



✗ **Limit Note:** No non-repudiation. Since both Sender and Receiver hold the exact same key, neither can cryptographically prove to a third-party judge who authored the message.

What a MAC Lacks



⚠ **Limit Note:** The Replay Attack: A valid, authenticated message is recorded and re-sent 50 times. The MAC verifies perfectly every single time because the cryptographic math checks out.

The Fix: Protocol-level state (sequence numbers, timestamps, nonces) is strictly required alongside the MAC to establish freshness.

[SECURE PRIMITIVE] + [INCORRECT COMPOSITION] ≠ [SECURE SYSTEM]

- AES is a secure primitive. CBC-MAC with variable lengths is an insecure system.
- SHA-256 is a secure primitive. $H(K || M)$ is an insecure system.

Cryptographic security is not an absolute trait; **it is a property of a precisely defined construction** operating within a strictly defined adversarial model.
Perfect gears assembled incorrectly will still shatter the machine.